

Jupyter Notebook Execution Report

Name: Rodrigo Tapiador Cano

Date: April 04, 2026

Cell 1: ■ Markdown

Ejercicios Prácticos – Curso de Python

****Objetivo:**** Este notebook contiene dos ejercicios diseñados para repasar y validar conocimientos fundamentales de Python, incluyendo funciones, estructuras de datos, manejo de errores y programación orientada a objetos.

Cell 2: ■ Markdown

Ejercicio 1: Análisis Básico de Datos de Ventas

Imagina que tienes los datos de ventas de una tienda en un formato de texto simple. Cada línea representa una venta y contiene `producto,cantidad,precio\_unitario`. Tu tarea es procesar estos datos para extraer información útil.

****Conocimientos a prueba:****

- * Lectura y manipulación de strings.
- * Funciones (definición, parámetros, retorno).
- * Estructuras de datos (listas y diccionarios).
- * Manejo de errores (`try-except`).
- * Bucles y condicionales.

Cell 3: ■ Code

```
# No necesitas leer un archivo, usaremos esta variable como si fuera el contenido de uno.

datos_ventas_texto = """
Laptop,2,1250.50
Mouse,5,25.00
Teclado,3,75.99
```

```
Monitor,ERROR,450.00
Laptop,1,1200.00
Webcam,4,50.25
Mouse,10,22.50
Teclado,1,75.99
||||
```

Cell 4: ■ Markdown

Tareas

****1.** Crea una función `procesar_linea(linea)`:

* Esta función debe recibir una línea de texto (ej. `"Laptop,2,1250.50"`).

* Debe separar los datos, convertir la cantidad a entero y el precio a flotante.

* Si una línea tiene un error (como "ERROR" en lugar de un número), la función debe manejarlo elegantemente usando un bloque `try-except` y devolver `None`.

* Si la línea es correcta, debe devolver un ****diccionario**** con las claves `producto`, `cantidad` y `precio`.

****2.** Procesa todos los datos:

* Crea una lista vacía llamada `ventas_procesadas`.

* Itera sobre cada línea de `datos_ventas_texto`. Llama a tu función `procesar_linea` por cada una y, si el resultado no es `None`, añádelo a la lista `ventas_procesadas`.

****3.** Calcula las métricas:

* ****Ventas Totales:**** Calcula el ingreso total de todas las ventas (suma de `cantidad * precio` para cada venta).

* ****Producto Más Vendido:**** Encuentra el nombre del producto que más unidades ha vendido en total.

Cell 5: ■ Code

```
# TU CÓDIGO AQUÍ

# 1. Función para procesar una línea
def procesar_linea(linea):
    try:
        lista = linea.strip().split(',')
        producto = str(lista[0])
        cantidad = int(lista[1])
```

```
precio = float(lista[2])
except Exception:
    return None
return {
    "producto": producto,
    "cantidad": cantidad,
    "precio": precio
}
```

```
# 2. Bucle para procesar todos los datos
ventas_procesadas = []
lineas = datos_ventas_texto.split("\n")
for linea in lineas:
    datos = procesar_linea(linea)
    if datos is not None:
        ventas_procesadas.append(datos)
```

```
# 3. Cálculo de métricas
producto_mas_vendido = ""
total_ingresos = 0
max_cantidad = 0
```

```
total_ingresos = sum(venta['cantidad'] * venta['precio'] for venta in
ventas_procesadas)
```

```
for venta in ventas_procesadas:
    if venta["cantidad"] > max_cantidad:
        producto_mas_vendido = venta["producto"]
        max_cantidad = venta["cantidad"]
```

```
#for venta in ventas_procesadas: total_ingresos += venta["cantidad"] *
venta["precio"]
#producto_mas_vendido = max(venta["cantidad"] for venta in ventas_procesadas)
```

```
# --- Imprime tus resultados ---
print(f"Ingresos Totales: ${total_ingresos:.2f}")
print(f"Producto más vendido: {producto_mas_vendido}")
```

Output:

Ingresos Totales: \$4555.96

Producto más vendido: Mouse

Cell 6: ■ Markdown

Ejercicio 2: Modelando el Inventario con OOP

Ahora, vamos a modelar los productos de nuestra tienda usando Programación Orientada a Objetos.

****Conocimientos a prueba:****

- * Clases y Objetos (``class``, ``__init__``).
- * Métodos de instancia.
- * Métodos especiales (``__str__``).
- * List Comprehension.

Cell 7: ■ Markdown

Tareas

****1. Crea una clase `Producto`:****

- * El constructor (``__init__``) debe recibir ``nombre``, ``precio`` y ``stock_inicial``.
- * Debe tener un método ``actualizar_stock(cantidad)`` que sume o reste del stock actual. El stock nunca puede ser negativo.
- * Debe tener un método ``valor_inventario()`` que devuelva el valor total de las unidades en stock (``precio`` * ``stock``).
- * Implementa el método especial ``__str__`` para que al imprimir un objeto ``Producto``, se muestre un resumen legible (ej: ``"Producto: Laptop | Precio: $1250.50 | Stock: 10 unidades"``).

****2. Gestiona un inventario:****

- * Crea una lista llamada ``inventario``.
- * Crea al menos 3 instancias diferentes de tu clase ``Producto`` y añádelas a la lista ``inventario``.
- * Usa un bucle para imprimir la información de cada producto en el inventario.
- * **** (Bonus) ****: Usa una ****List Comprehension**** para crear una nueva lista llamada ``productos_poco_stock`` que contenga solo los ****nombres**** de los productos cuyo stock sea inferior a 5 unidades.

Cell 8: ■ Code

```
# TU CÓDIGO AQUÍ

# 1. Definición de la clase Producto

class Producto:
    def __init__(self, nombre, precio, stock_inicial):
        self.nombre = nombre
        self.precio = precio
        self.stock = stock_inicial

    def actualizar_stock(self, cantidad):
        nuevo_stock = self.stock + cantidad
        if nuevo_stock >= 0:
            self.stock = nuevo_stock
        else:
            print("No tenemos stock para cubrir esa cantidad")

    def valor_inventario(self):
        return self.precio * self.stock

    def __str__(self):
        return f"Producto: {self.nombre} | Precio: ${self.precio:.2f} | Stock: {self.stock} unidades"

# 2. Gestión del inventario

inventario = []
inventario.append(Producto("Laptop", 1250.50, 10))
inventario.append(Producto("Teclado Mecánico", 85.00, 3))
inventario.append(Producto("Mouse Inalámbrico", 45.99, 25))
inventario.append(Producto("Monitor 4K", 450.75, 4))
inventario.append(Producto("Webcam Full HD", 70.00, 1))

# Imprimir inventario
for producto in inventario:
    print(producto)

# (Bonus) List comprehension para productos con poco stock
productos_poco_stock = [p.nombre for p in inventario if p.stock < 5]
```

```
print("\n--- Productos con poco stock (&lt; 5 unidades) ---")
print(productos_poco_stock)
```

Output:

```
Producto: Laptop | Precio: $1250.50 | Stock: 10 unidades
Producto: Teclado Mecánico | Precio: $85.00 | Stock: 3 unidades
Producto: Mouse Inalámbrico | Precio: $45.99 | Stock: 25 unidades
Producto: Monitor 4K | Precio: $450.75 | Stock: 4 unidades
Producto: Webcam Full HD | Precio: $70.00 | Stock: 1 unidades

--- Productos con poco stock (&lt; 5 unidades) ---
['Teclado Mecánico', 'Monitor 4K', 'Webcam Full HD']
```

Cell 9: ■ Code